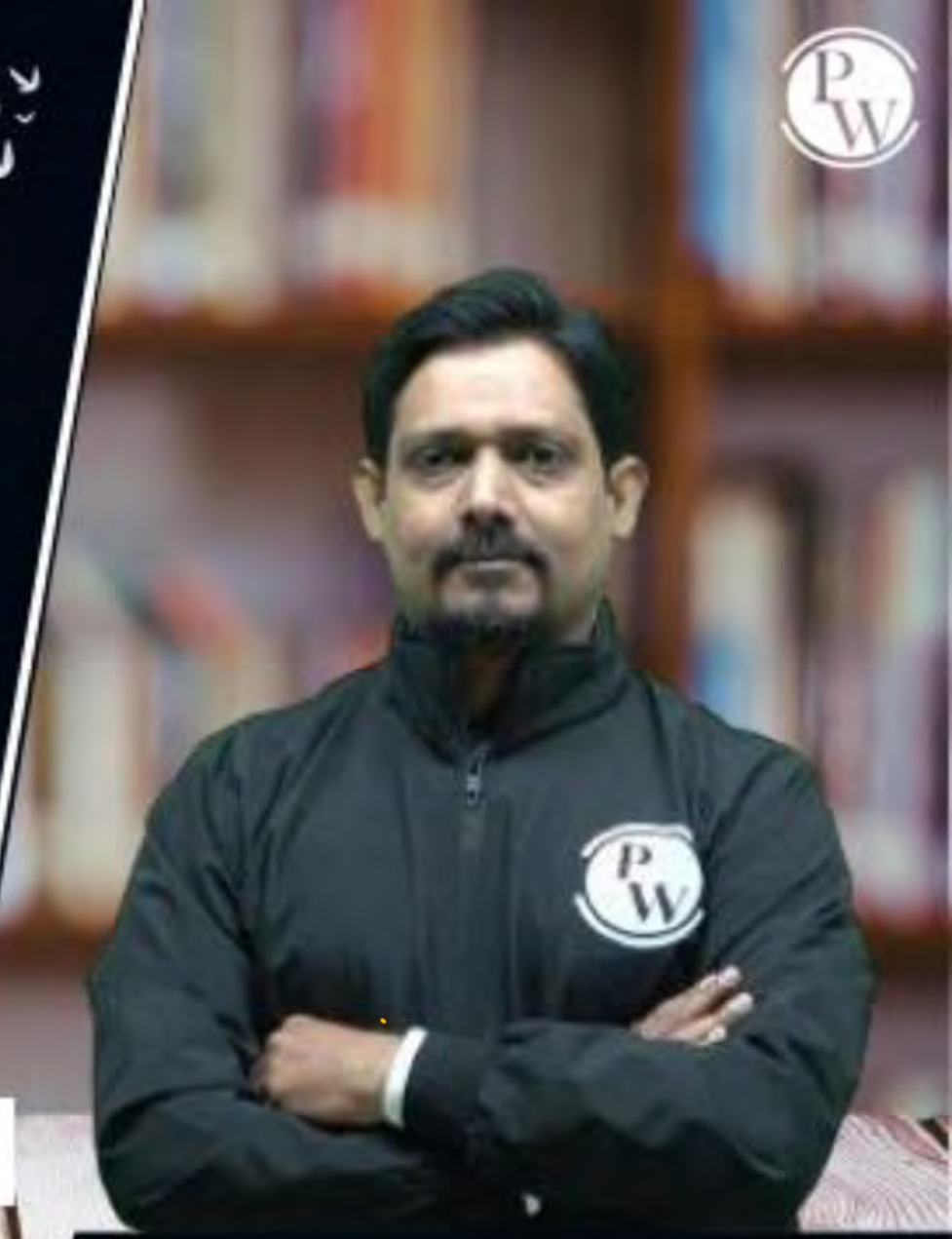
Computer Science & IT



Data Structure & Programming

Stack

Lecture No. 03



By- Abhishek Sir

Recap of Previous Lecture



Topic	Stack as Permutation Generator
Topic	Expression In computer
Topic	Practice Problem
Topic	
Topic	

Slide

Topics to be Covered



- Topic — Expression without stack
- Topic — stack;
- Topic — practice problem
- Topic — Evaluation of postfix/prefix
- Topic

Slide



Topic : Expression In Computer

Bracket	()		Highest
Unary minus	-		
Exponentation	↑or $ or ^	Right to Left	
Multiplicative	*, /, %	Left to Right	
Additive	+ -	Left to Right	
Equal	=	Right to Left	Lowest

Unary

↑ $ ^

Slide



Topic : Expression In Computer

a + b a, b, 5, 6, operand ← Infix

5 + 6 + operator

prefix : operator operand, operand$_2$

+ a b , + 5 6

postfix : operand$_1$ operand$_2$ operator

a b + . 5 6 +

Slide



Topic : Expression In Computer

(a + b) - c

(a - b) + c

(a + b) + c

(a - b) - c

(a / b) / c

(a * b) / c

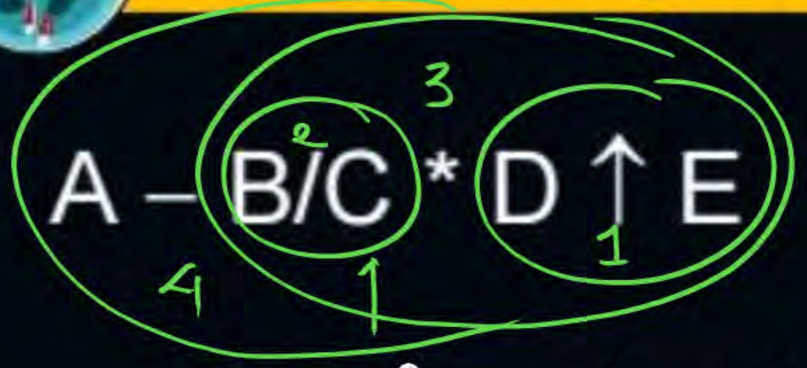
(a / b) * c

(a * b) * c

Slide



Topic : Expression In Computer



operand

postfix := A - B/C * DE↑

= A - BC/ * DE↑

= A - BC/ DE↑ *

= ABC/DE↑*-

operand

poefix

A - B/C * ↑DE

= A - /BC * ↑DE

= A - * /BC ↑DE

= - A * /BC↑DE

poefix

Slide

Topic : Expression In Computer

$ABC*+D-EF\uparrow-G+$

$A + (B * C) - D - (E \uparrow F) + G$

(3) (2) (4) (5) (1) (6)

$= A + B*C - D - EF\uparrow + G$

$= A + BC* - D - EF\uparrow + G$

$= ABC*+ - D - EF\uparrow + G$

$= ABC*+D- - EF\uparrow + G$

$= ABC*+D-EF\uparrow- + G$

prefix :

$= A + *BC - D - \uparrow EF + G$

$= +A*BC - D - \uparrow EF + G$

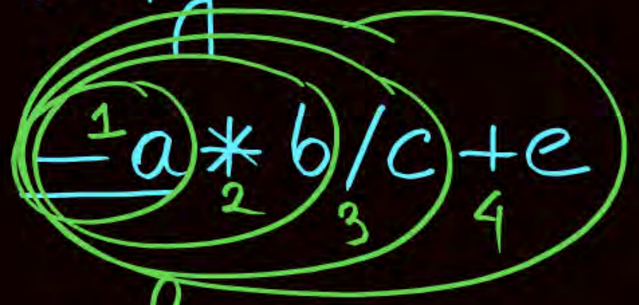
$= -+A*BCD - \uparrow EF + G$

$= --+A*BCD\uparrow EF + G$

$= +--+A*BCD\uparrow EFG$

Slide

Unary −



prefix :

$-a * b/c+e$

$*-ab/c+e$

$/*-abc+e$

$+/*-abce$

$-a$ prefix $-a$ $-a$ postfix $a-$

postfix :

$a- * b/c+e$

$= a-b* /c+e$

$= a-b*c/ +e$

$= a-b*c/e+$ postfix expression



Topic : Expression In Computer

$$A + B / C - D \uparrow E \uparrow F + G$$

Slide



Topic : Question

2004

#Q. Assume that the operators + , – , ×, are left associative and ^ is right associative. The order of precedence (from highest to lowest) is ^, x, +, – . The postfix expression corresponding to the infix expression a + b × c – d ^ e ^ f is

(A) abc × + def ^ ^ –

(B) abc × + de ^f ^ –

(C) ab + c × d – e ^ f ^

(D) – + a × bc ^ ^ def

$(d \wedge (e \wedge f)) = a + bc\times - d \wedge ef \wedge$

$= a + bc\times - def \wedge\wedge$

$= abc\times + - def \wedge\wedge$

$\boxed{abc\times + def \wedge\wedge -}$

Slide



Topic : Expression In Computer

using stack Infix to postfix

1. operator stack is maintained
2. Scan Input left to right
3. printing (output) left to right
4. if operand is Input print the operand.
5. if operator is Input and stack is empty push the operator in stack.
6. If Input is operator and having higher precedence than operator on top of stack the peform push the operator. **HPU**
7. If Input is operator and having lower preccedence than operator on top of stack. **LPoPu**

pop operators until Input operator having lower precedence. push the New operator

Slide

Topic : Expression In Computer

5+6*7 ⇒ 5+67*

567*+

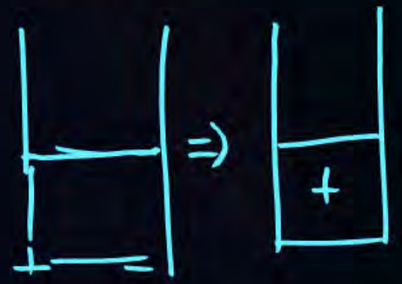
(1) Input 5

output: 5



(2) Input : +

output : 5



(3) Input : 6

output : 56



4. Input : *

output : 56



Input * having higher proecedece than + the perform push

HPU

5 Input : 7

output : 567



6. End of input : pop evary thing & print

567*+

Slide

Topic : Expression In Computer

5*6-7

1. Input : 5
   output : 5



2. Input *
   output : 5



3. Input : 6
   output : 56



4 Input : —
output : 56



push New operation

56*
pop

5. Input : 7
   output 56*7



6. End of Input, pop everything & print
   output: 56*7-

L

Lpopspu

5*6-7
=56*-7 = 56*7-

Slide

Topic : Expression In Computer

5 ↑ 6 * 7 - 4 (stack)

* (LPoPu) — LPoPu



5 6 ↑ 7 * 4 -

5 6 ↑ * 7 - 4

5 6 ↑ 7 * - 4

5 6 ↑ 7 * 4 -

end of Input 5 6 ↑ 7 * 4 -

pop everything 2 print

Slide



Topic : Expression In Computer

5 * 6 - 7 prefix

1. Scan Right to Left
2. print Right to left

3. Empty stack operator is Input (Same as postfix)
4. Operand is Input : Same as postfix
5. Higher precedece operator than Top of stack (HPU)
6. Lower precedence operator than Top of stack (LPoPu)

Reverse the string →
follow postfix Rule
Reverse the expression

7 - 6 * 5

7 - 65 *

7 65 * -

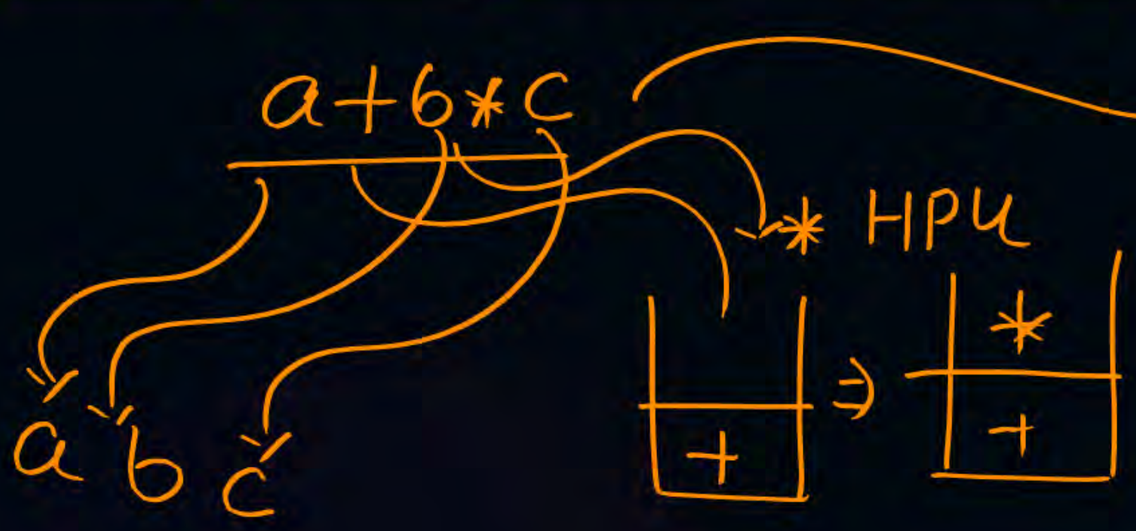
Reverse

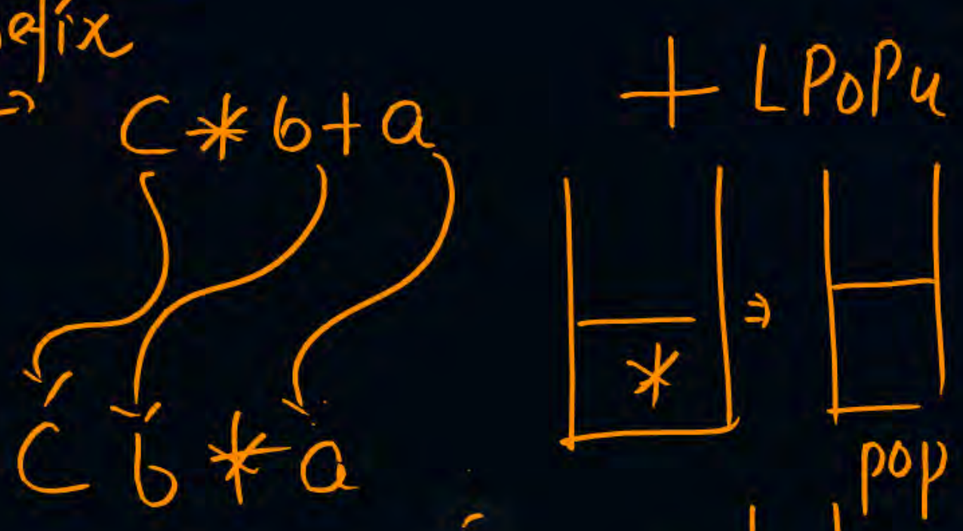
- * 5 6 7

prefix

Slide

Topic : Expression In Computer





abc*+

a+b*c

a1 *bc

+a*bc

cb*a+

+a*bc

Slide



Topic : Stack Application

(i) The value of Fibonacci (7) is _____

(ii) The total number of calls in Fibonacci(8) is _____

(iii) The total no. of '+' in Fibonacci (9) is_____

$f_{16}(7)$ value Calculation Recursion tree should Not used

Bottom up

$f(0) = 0$

$f(1) = 1$

$f_{16}(2) = 1$

$f_{16}(n) = f_{16}(n-1) + f_{16}(n-2)$

$f_{16}(3) = 2$

$f_{16}(4) = 3$

$f_{16}(5) = 5$

$f_{16}(6) = 8$

$f_{16}(7) = 13$ ✓

value

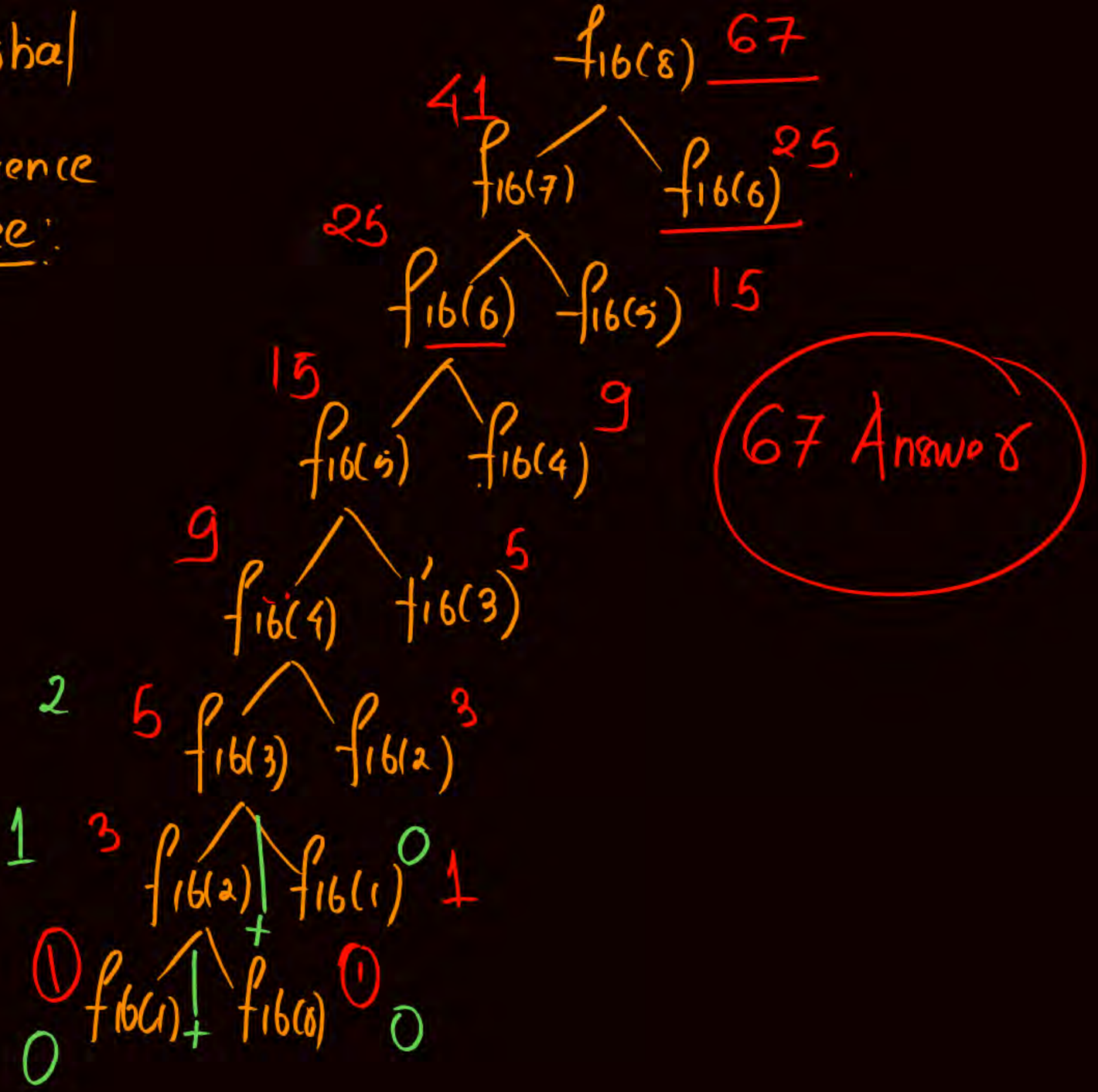
* Bottom up calculation

Smaller → Large

Recurrence Relation should be used.

fib(8) No. of calls

pasbal Recurrence Tree:



67 Answer



2 mins Summary

Topic

Topic

Topic

Topic

Topic

Expression prefix Postfix

Conversion using stack

Slide

THANK - YOU

Slide